Non - Formatted Input Output

In C programming, **gets**, **puts**, **getchar**, and **putchar** are standard library functions found in the <stdio.h> header file used for handling input and output without formatting specifiers (like %d or %s). They are divided into character-based operations and string-based operations.

1. Character-Based Functions: getchar() & putchar()

These functions read and write **one single character** at a time.

getchar()

• **Purpose:** Reads the next available character from the standard input (stdin, usually the keyboard).

• **Return Type:** It returns an int (integer). This allows it to return all possible character values as well as the special EOF (End-of-File) marker.

• **Syntax:** int ch = getchar();

putchar()

• **Purpose:** Writes a single character to the standard output (stdout, usually the terminal screen).

• **Syntax:** putchar(ch);

Code Example:

```
#include <stdio.h>

int main() {
    int c;
    printf("Type a character and press Enter: ");

    c = getchar(); // Reads one character
    printf("You typed: ");
    putchar(c);    // Prints that character
    putchar('\n'); // Prints a newline

    return 0;
}
```

Example Output:

```
Type a character and press Enter: Hello
You typed: H
```

Note: Why did it only print 'H'? getchar() only reads the very first character available in the input buffer. The rest of the letters ('ello') are left behind unread.

2. String-Based Functions: gets() & puts()

These functions are designed to handle **entire lines of text (strings)**.

gets()

• **Purpose:** Reads a line of text from stdin into a character array until it hits a newline character or EOF.

• ■■ **Critical Warning: gets() is highly unsafe and obsolete.** Because it does not know the size of the destination array, typing a string that is too long will cause a **buffer overflow**. It was officially removed from the C programming language standard. **You should use fgets() instead**.

puts()

• **Purpose:** Writes a string to stdout and **automatically appends a newline character (\n)** to the end.

Code Example (Using safe alternative fgets):

```
#include <stdio.h>

int main() {
    char str[100]; // Buffer for up to 99 characters + \0
    printf("Enter a line of text: ");

    // fgets is the safe modern replacement for gets()
    fgets(str, sizeof(str), stdin);

    printf("You entered: ");
    puts(str); // Prints the string and automatically adds a newline

    return 0;
}
```

Example Output:

```
Enter a line of text: Learning C Programming is fun!
You entered: Learning C Programming is fun!
```

Note: Why use this instead of scanf? scanf("%s", str) stops reading at the first blank space, only capturing 'Learning'. fgets() and puts() capture the entire sentence including spaces.

Real-World Usecases

• **getchar() & putchar():** Used when you need precise, low-level control over text processing (e.g., compilers, custom command line parsers, interactive menu keys).

• **fgets() & puts():** Used for line-by-line text handling (e.g., reading configuration files, processing structured text files, taking descriptive inputs with spaces).

Are They Widely Used in C Today?

• **getchar() & putchar():** Yes. Highly efficient and safe for stream-based character parsing.

• **puts():** Yes. Simpler, faster alternative to printf() when printing static text lines.

• **gets():** Banned. Completely removed from the official C specification due to buffer overflow flaws.

• **fgets():** Yes. The modern industry standard for reading strings safely in C.

Quick Comparison Table

Function	Type	Operates On	Auto Newline?	Safety Note
getchar()	Input	Character	No	Safe to use.
putchar()	Output	Character	No	Safe to use.
gets()	Input	String	No	■ Dangerous (Obsolete)
puts()	Output	String	Yes	Safe to use.